Choosing the Allocation solution

When it comes to the allocation problem we had to choose an appropriate solution for our use case, as we have several hard constraints to abide by and several desired soft constraints to optimise as much as possible.

We considered several solutions and the final decision was to choose a CP-SAT solver. We list the following options and clarify why each was considered and then rejected:

1. The Greedy Algorithm:

A simple algorithm that iterates through the list and picks the "best available" option right now, never looking back. It was considered because it's incredibly easy to code and debug and it runs instantly.

- **Why we refused it:** it's unreliable when it comes to optimising soft constraints and it is highly prone to local optima because it doesn't see the full picture. It would greedily assign all the flexible teachers for easy cases and optimise the easy soft constraints. Later, when it encounters a "Hard" slot (SFP-Math in Zone 1), no feasible solution remains

CP-SAT searches **globally**. It "sees into the future" and saves the flexible teachers for the hard slots.

2. Genetic Algorithms (GA) / Evolutionary Search

An approach that generates random solutions as populations, "breeds" the best individuals in the population, and "mutates" them to find a good result over thousands of generations.

It is excellent for Multi-Objective Optimization. It handles soft constraints (like "Diversity" and "Balance") naturally and can produce "Good Enough" solutions quickly for messy problems.

- **Why we refused it:** It would be unsuitable with the Hard Constraints.

  The problem is that we have several hard constraints to satisfy so designing a valid mutation operator would be very tricky. In our system a lot of the random changes create an Illegal State (e.g., assigning a Zone 3 teacher to an SFP). The GA would waste much of its computation time just trying to find a legal schedule, let alone an optimal one.

  Another reason is inconsistency. Every time we run it we would get a completely different plan. The client would hate this.

CP-SAT uses Constraint Propagation. It mathematically *guarantees* that it never even considers an illegal state, making it exponentially more efficient for highly constrained problems. In addition, unlike genetic algorithms, which can produce completely different plans on each run, the solver uses constraint propagation and objective-guided search to produce consistent, predictable schedules, especially because it maintains a global view of all possible solutions

3. Mixed Integer Programming (MIP) (e.g., Gurobi, CBC)

It's the industrial standard for operations research. It models everything as linear algebraic equations. It is mathematically proven to find the **Optimal** solution (not just a "good" one). It would handle the budget resource allocation flawlessly.

- **Why we refused it:** It struggles with **Complex Boolean Logic**.

  The Problem: MIP solvers cannot understand logic like OR or IF. For example, to model "Valid Pair = (A+B) OR (C+D)", we have to use a hack called "Big-M Constraints." We would create extra "Indicator Variables" (z_1, z_2) and write equations like $x_A + x_B - 2z_1 \leq 0$. This will cause our code to bloat from 200 lines to 1000 lines of unreadable matrix algebra. It becomes incredibly hard to debug logic errors

  In addition, standard MIP solvers are slower at handling these "Logical" constraints than CP solvers. They have to "branch" on every single indicator variable, exploding the search tree.

In contrast, CP-SAT is a **hybrid**. It handles the counting (Capacity) like an MIP solver, but it handles the logic (Valid Pairs) natively with Boolean gates (OnlyEnforceIf). It gives us the power of MIP with the expressiveness of Python logic.

4. Constraint Programming with SAT (CP-SAT via Google OR-Tools)

This is a modern hybrid solver that combines the arithmetic power of Integer Programming with the logical expressiveness of SAT solvers.

Why we chose it:
It is the only solution that natively handles the dual nature of our problem:

1. **The "Arithmetic" Problem:** It efficiently handles requirements like counting resources (Sum of Assignments <= Capacity) using Linear Constraints, similar to MIP.
2. **The "Logical" Problem:** Requirements like structuring schedules (e.g. If Mentor is Full then Assignments must match a Valid Pair) is handled using Boolean Logic and Implication (OnlyEnforceIf), similar to SAT.

Unlike Greedy algorithms, it guarantees a global search that avoids local optima. Unlike Genetic Algorithms, it uses **Constraint Propagation** to prune the search tree, meaning it never wastes time exploring invalid solutions.

It allowed us to write readable, maintainable Python code that expresses complex business rules directly, resulting in a solver that is both **mathematically optimal** and **structurally robust**.

Allocation Algorithm Implementation

The implemented allocation system is a **Constraint Satisfaction & Optimization Engine** built using Google OR-Tools (CP-SAT). It operates on a **Supply-Driven Model** (Capacity Maximization), focusing on utilizing the available mentor budget (Anrechnungsstunden) efficiently while adhering to strict academic and geographical constraints.

Phase 1: Pre-Processing (Data Preparation)

Before the solver is initialized, raw data is filtered and transformed to define the valid "Search Space."

Supply Analysis & Eligibility Calculation

Source: assignments/services.py

For every active mentor, the system calculates a list of "Legal Moves" (valid assignments). A move is valid only if it passes **all** of the following checks:

1. **Status Check:** Mentor must be is_active and not marked as unavailable in notes.
2. **Subject Qualification:** For SFP/ZSP, the mentor must have the specific subject marked in the database.
3. **Curriculum Validity:** The subject must be valid for the school type (e.g., Arbeitslehre is valid for MS, but invalid for GS).
4. **Reachability (Hard Constraint):** SFP/ZSP (Wednesday-praktika) assignments are **strictly forbidden** in Zone 3. They are allowed only in Zone 1 or 2. PDP (Block-praktika) assignments are allowed in all zones.

Phase 2: The Solver Model (Hard Constraints)

Source: assignments/solver.py and assignments/constraints.py

Variables are created for every valid (Mentor, Type, Subject) combination. The solver must select a subset of these variables while obeying the following immutable laws:

A. Capacity Constraint

- **Rule:** A mentor cannot be assigned more slots than their budget allows (Capacity = Anrechnungsstunden * 2).
- **Implementation:** sum(assignments) <= Capacity.
- **Note:** We use <= (Less Than or Equal) instead of == to prevent the solver from crashing if a mentor has insufficient eligible options to fill their quota.

B. Valid Pairs Constraint (Structural)

- **Rule:** Assignments must follow academic structure (e.g., PDP I + SFP is valid; SFP + SFP is invalid).
- **Condition:** This rule is only enforced if the mentor reaches 2 slots. So mentors with an Anrechnungsstunden larger then 1 (meaning requires more than 2 slots) do not abide to this rule.

C. Minimum Coverage Constraint (Granular)

- **Rule:** Every unique combination of (PracticumType, Program, Subject) that is mathematically possible (has at least one eligible mentor) **MUST** be assigned at least once.
- **Goal:** Guarantees that niche subjects are not completely ignored in favor of core subjects.

Phase 3: The Objective Function (Soft Constraints)

Source: assignments/objectives.py

The solver treats the scheduling challenge as a **Multi-Objective Optimization Problem (MOOP)**. Rather than relying on simple business logic, the system constructs a fitness landscape using **Weighted Sum Aggregation**. This approach combines competing objectives—such as stability, diversity, and scarcity—into a single scalar utility score to guide the search toward the global optimum.

The maximization objective is defined as:

$$Maximize\ Z = \sum_{i=1}^{n} (x_i \cdot w_i)$$

Where x_i represents the assignment variable and w_i represents the associated weight

Weight Name	Value	SBSE Purpose / Business Logic
Utilization	100	**Base Heuristics.** Drives the solver away from trivial (empty) solutions to use the budget.
Diversity	70	**Clustering Prevention.** Rewards schools for hosting different internship types.
Subject Granularity	45	**Semantic Density Maximization**. Rewards specific subject codes over generic "N/A" (PDP).
Constraint Tightness	30	**Resource Preservation.** Incentivizes flexible mentors to take restricted slots.
Continuity	70	**Stability.** Bonus if the assignment matches the mentor's history S_{t-1}.
Mixed Type	20	**Structural Motif.** Bonus for assigning mentors a mix of Block + Wednesday internships.
Core Subject	15	**Bias Injection.** Boosts priority for core subjects (e.g. Math, German).
Scarcity	Dynamic	**Dynamic Heuristic.** Prioritizes assigning rare subjects over common ones.
Soft Cap	-70 (per violation)	**Resource Levelling.** Penalty if a subject exceeds its slot limit.
Same Subject	-35	**Constraint Relaxation.** Penalty to discourage assigning a mentor to the same subject twice.

1. **Base Heuristics (Utilization)**

This Drives Search from Trivial Solutions By making it "profitable" for the solver to use capacity, we prevent the search from stagnating at a null assignment (an empty schedule), which represents a local optimum of zero cost but zero utility.

- **Heuristic Magnitude Scaling:** The value of **100** acts as the **Baseline Fitness**. It is carefully tuned to establish a "primary gradient" in the fitness landscape. This ensures that "Quantity" (Utilization) provides the foundation of the score, while "Quality" weights—such as Diversity (+70) and Continuity (+70)—are weighted strongly enough to influence *which* slots are filled without overriding the primary goal of budget exhaustion.
- **Budget Consumption as an Objective:** The system treats the mentor's total available capacity as a resource to be maximized. This transforms the business requirement of "using the budget" into a mathematical pressure that pushes the solver toward the edges of the feasible search space.

2. **Dynamic Heuristics (Scarcity)**
   he score for a subject-specific assignment is inversely proportional to how many mentors can supply that subject:(100 / supply_count) * SCARCITY_WEIGHT with SCARCITY_WEIGHT = 10, **capped at 50**. So the effective bonus is min((100 / supply_count) * 10, 50). This prevents the solver from converging on "easy" resources (e.g., common subjects) and forces the search to secure hard-to-fill resources (e.g., IT) early in the computation.
3. **Diversity Preservation (School Balance)**
   Optimization algorithms often suffer from "clustering," where they assign similar tasks to a single group. By applying a "Diversity" bonus using a boolean step function, the objective function rewards solutions that increase diversity within a school's assignment set, encouraging a spread of internship types.
4. **Temporal Stability (Continuity)**
   To minimize the administrative "distance" between the previous year's solution (S_{t-1}.) and the current solution (S_t), a "Continuity" bonus is applied. This creates temporal stability, ensuring the solver does not generate a plan that is mathematically optimal but administratively chaotic.
5. **Structural Motifs & Bias (Mixed Type & Core Subjects)**
   User-defined preferences when it comes to subject prioritization are taken into account and are injected directly into the fitness landscape. The "Mixed Type" bonus rewards specific high-quality structural patterns—in our case, mentors supervising both Block and Wednesday internships, creating solutions that are structurally robust rather than just numerically full.
6. **Constraint Relaxation & Resource Levelling (Penalties)**
   Instead of hard constraints, which can render problems infeasible, **Soft Constraints** are used to model undesirable states.
   - **Same Subject:** Assigning a mentor the same subject twice results in lower fitness, naturally guiding the solver toward higher variety.
   - **Soft Caps:** This implements a piecewise linear penalty functionto model diminishing returns (Each unit of excess adds **-70** to the objective). It forces the solver to stop exploiting one resource (oversupply of a core or niche subject) and explore others, effectively leveling resources across the Pareto front. The cap for a core subject is 30 slots while the cap for the normal subjects is 20 slots

7. **Strategic Slot Prioritization ((Semantic Density & Feasibility)** Two specific heuristics function together to bias the solver toward SFP and ZSP assignments, addressing two distinct optimization challenges:
   A. **Solution Granularity (Semantic Density Maximization):** Specific qualifications (e.g., "Math") have higher intrinsic administrative value than generic roles. By adding a **+45 reward** where subject_code != "N/A", we increase the semantic density of the solution. This prevents the solver from taking the easy path of filling capacity with generic PDP slots.
   B. **Constraint Tightness (Resource Preservation):** To solve the "Zone 3 Trap," we use **Most Constrained Variable Logic**. Zone 1 mentors are "flexible" (valid for all types), while Zone 3 mentors are "constrained" (valid only for PDP). By adding a **+30 reward** for SFP/ZSP types, we incentivize flexible resources to consume the "tightest" slots. This preserves the "loose" PDP slots for the Zone 3 mentors who have no other legal moves, ensuring the entire budget remains usable.

Phase 4: Output & Persistence
Source: assignments/solver.py
When the solver finds an optimal or feasible solution:

- **Transaction:** Persistence and reporting run inside a single database transaction.
- **Clean slate:** All existing Assignment rows are deleted. All Student rows with placement_status="PLACED" are set to "UNPLACED".
- **Persistence:** New Assignment records are created for every variable set to 1 by the solver. Foreign Keys are resolved using pre-fetched caches.
- **Reporting:**
  - **Assignments:** A list of created assignments (each with mentor name, practikum type, subject) is returned.
  - **Unassigned report:** A list of mentors who are either fully unassigned (no assignments) or partially assigned (fewer assignments than capacity). Each entry includes id, name, email, reason, and school. Reasons are: "Solver Optimization (Leftover)", "DATA ERROR: No valid eligibilities found." for skipped mentors, or "Partial Assignment: X/Y slots filled. Missing Z." for under-capacity mentors. This list is used by the Manual Adjustment UI.

The implemented system successfully transforms a manual, heuristic process into a **mathematically rigorous optimization task**. It robustly handles data errors (via permissive validation), enforces complex legal/geographical rules (via Hard Constraints), and balances competing business goals (via the Multi-Objective Fitness Function).

Phase 5: API Layer & System Integration
Source: assignments/views.py and assignments/serializers.py
The optimization engine is exposed via a RESTful API (Django REST Framework) to ensure clean separation between the backend logic and the frontend interface.

- **Demand Preview:** The DemandPreviewAPIView provides a "pre-flight" check using the DemandPreviewSerializer. This calculates summary metrics (Supply vs. Demand) before the allocation is run, allowing administrators to foresee potential bottlenecks.
- **Solver Execution:** The SolverRunAPIView triggers the run_solver routine. It returns the structured SolverResultSerializer data, which includes the list of assignments and the granular "Unassigned Report" mentioned in Phase 4.

- **Export Services:** Dedicated endpoints (ExportAssignmentsExcelAPIView and ExportAssignmentsPDFAPIView) utilize openpyxl and reportlab to serialize the final assignment table into downloadable formats for administrative use.
- **Manual Adjustments:** The AssignmentUpdateAPIView allows for post-allocation overrides. It permits administrators to manually modify specific assignments while the system validates these changes against basic integrity rules (e.g., referential integrity for Schools and Subjects).